EXPERIMENT - 01

A PYTHON PROGRAM TO IMPLEMENT UNIVARIATE, BIVARIATE AND MULTIVARIATE REGRESION

Aim:

Develop a Python program to implement **univariate, bivariate, and multivariate regression** models and analyze the relationship between dependent and independent variables.

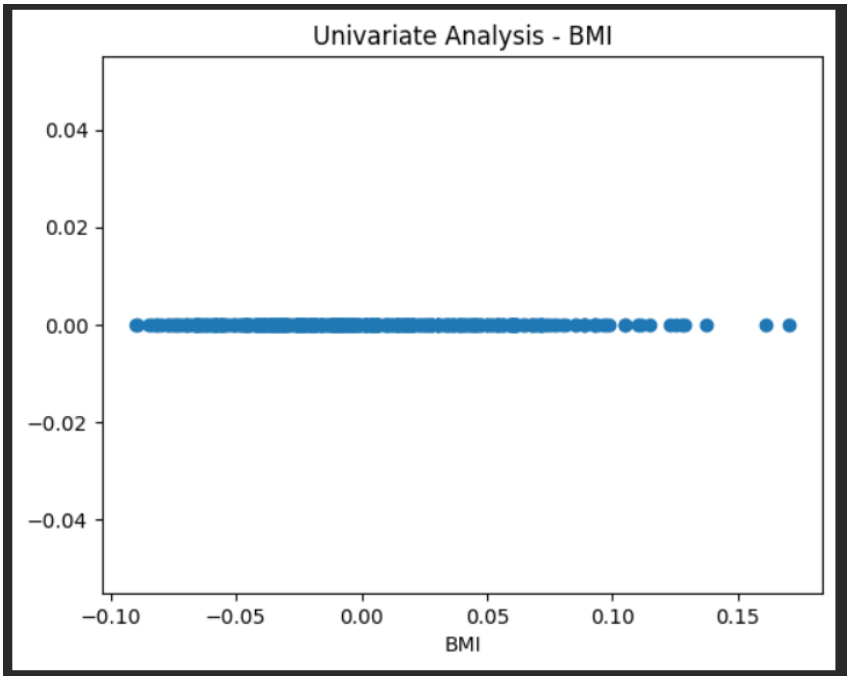
Requirements:
Jupyter Notebook with Python, Diabetes.csv dataset, and required Python libraries.

PROGRAM:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from sklearn.datasets import load_diabetes

# Load dataset
data = load_diabetes(as_frame=True)
df = data.frame

# Rename target column
df.rename(columns={"target": "disease_progression"}, inplace=True)

# Show dataset
df.head()
df.shape
```

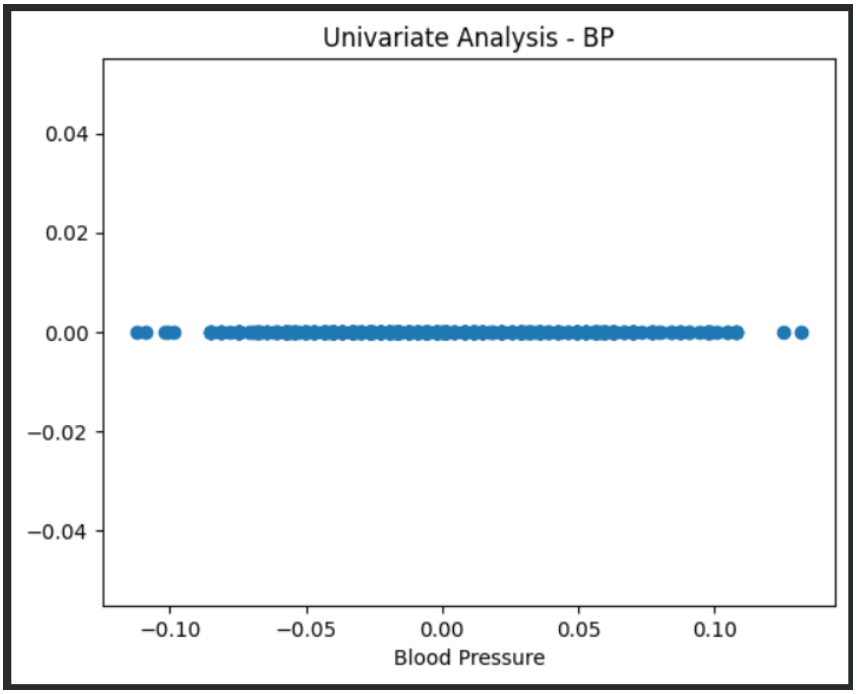
```
(442, 11)
```

```
#Univariate for BMI

plt.scatter(df['bmi'], np.zeros_like(df['bmi']))
plt.xlabel('BMI')
plt.title("Univariate Analysis - BMI")
plt.show()
```



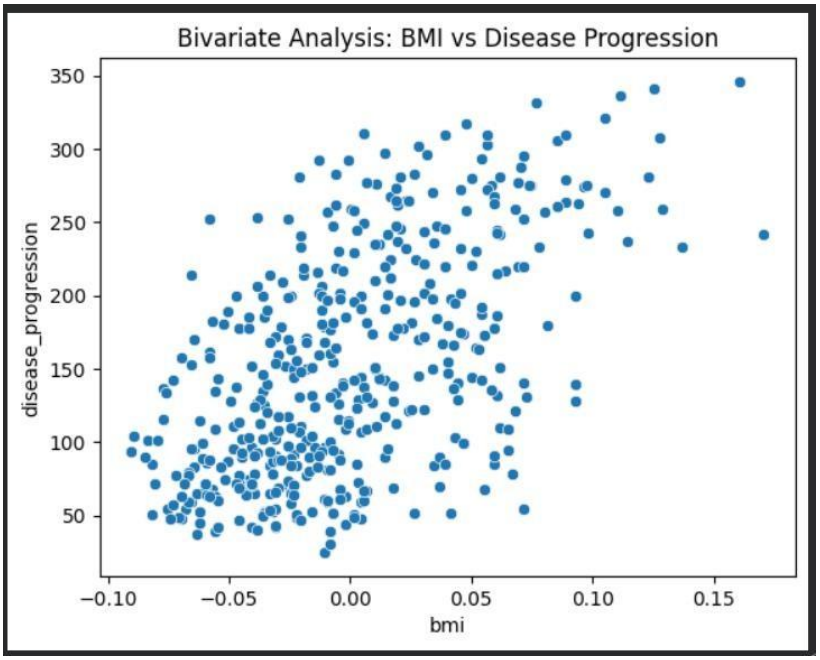
#univariate for BP

```
plt.scatter(df['bp'], np.zeros_like(df['bp']))
plt.xlabel('Blood Pressure')
plt.title("Univariate Analysis - BP")
plt.show()
```



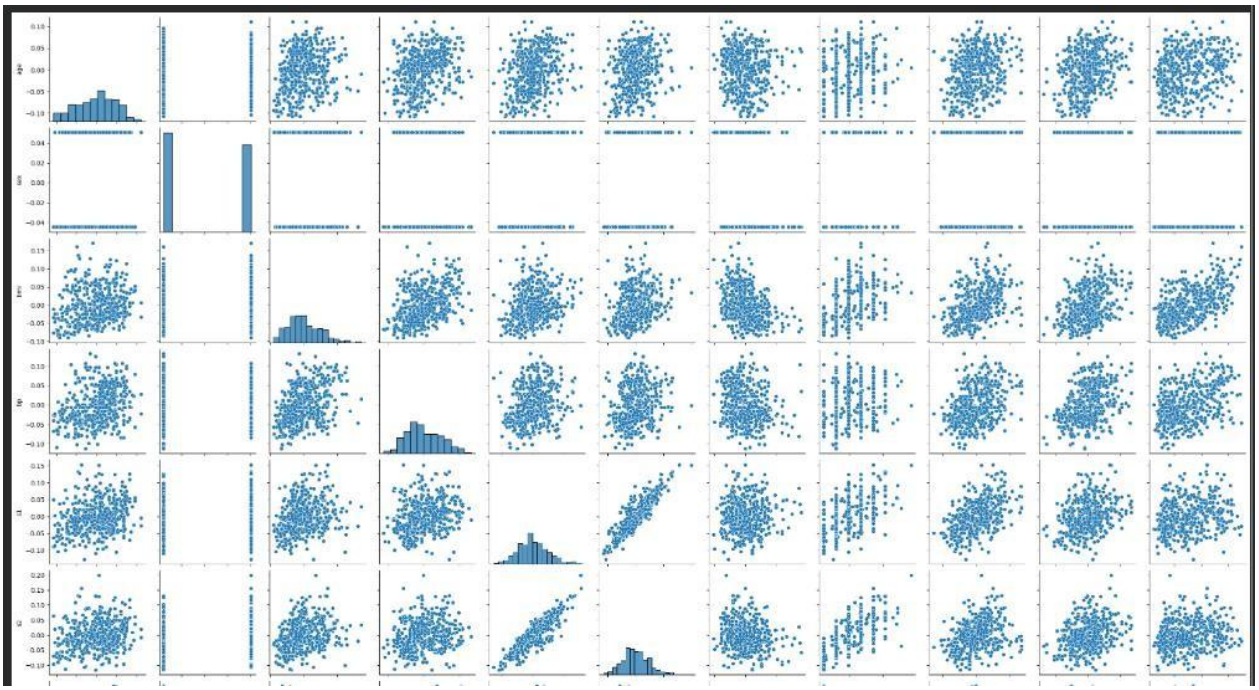
 # Bivariate Analysis: BMI vs Disease Progression

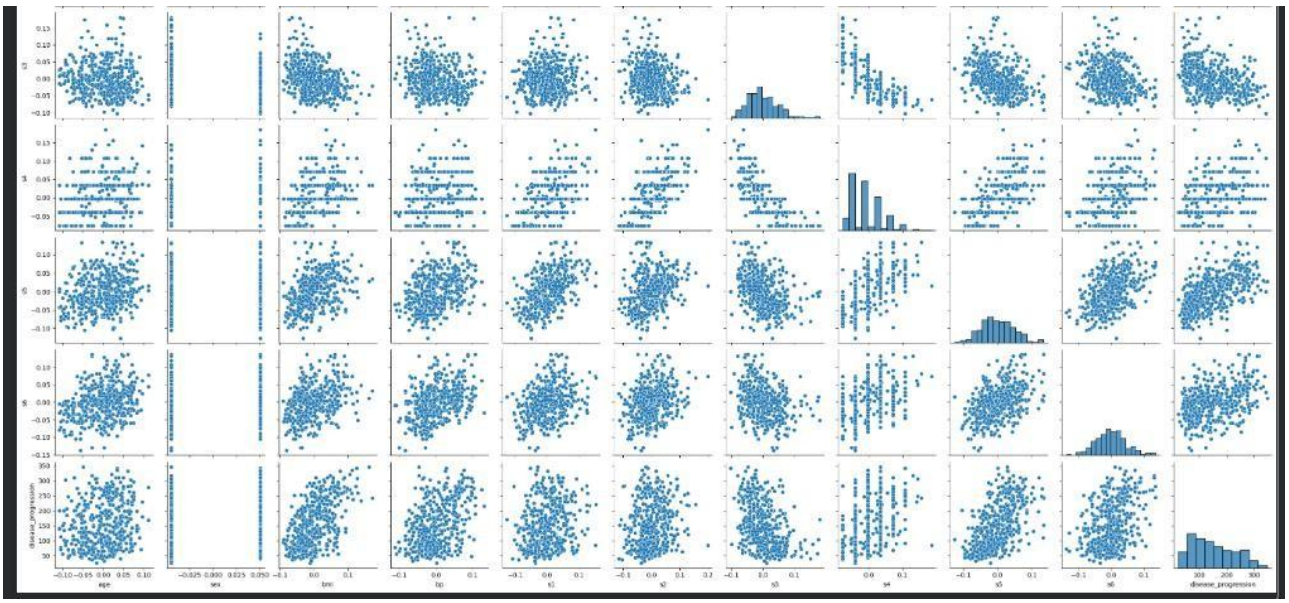
```
sns.scatterplot(x='bmi', y='disease_progression', data=df)
plt.title("Bivariate Analysis: BMI vs Disease Progression")
plt.show()
```



 # multivariate all the features

```
sns.pairplot(df)
plt.show()
```





Result:

The Python program for univariate, bivariate, and multivariate regression was successfully implemented using the Diabetes.csv dataset in Jupyter Notebook, and the regression models were able to analyze and predict relationships between variables accurately.

EXPERIMENT: 2

SIMPLE LINEAR REGRESSION USING LEAST SQUARE METHOD

Aim:

To implement Simple Linear Regression using the Least Squares Method in Python to analyze the relationship between Playing Hours (independent variable) and Performance Score (dependent variable).

Procedure:

1. Create a CSV dataset named `playing_data.csv` containing:
    - PlayingHours (independent variable)
    - Performance (dependent variable)
2. Import the required Python libraries:
    - pandas (for data handling)
    - numpy (for mathematical calculations)
    - matplotlib (for visualization)
3. Load the dataset using pandas.
4. Extract independent variable (x) and dependent variable (y).
5. Calculate the mean of x and y.
6. Compute the slope (b_1) using the formula:

$$b_1 = \frac{\sum(x - \bar{x})(y - \bar{y})}{\sum(x - \bar{x})^2}$$

7. Compute the intercept (b_0) using:

$$b_0 = \bar{y} - b_1\bar{x}$$

8. Form the regression equation:

$$y = b_0 + b_1 x$$

9. Calculate predicted values.
10. Plot the scatter graph and regression line.

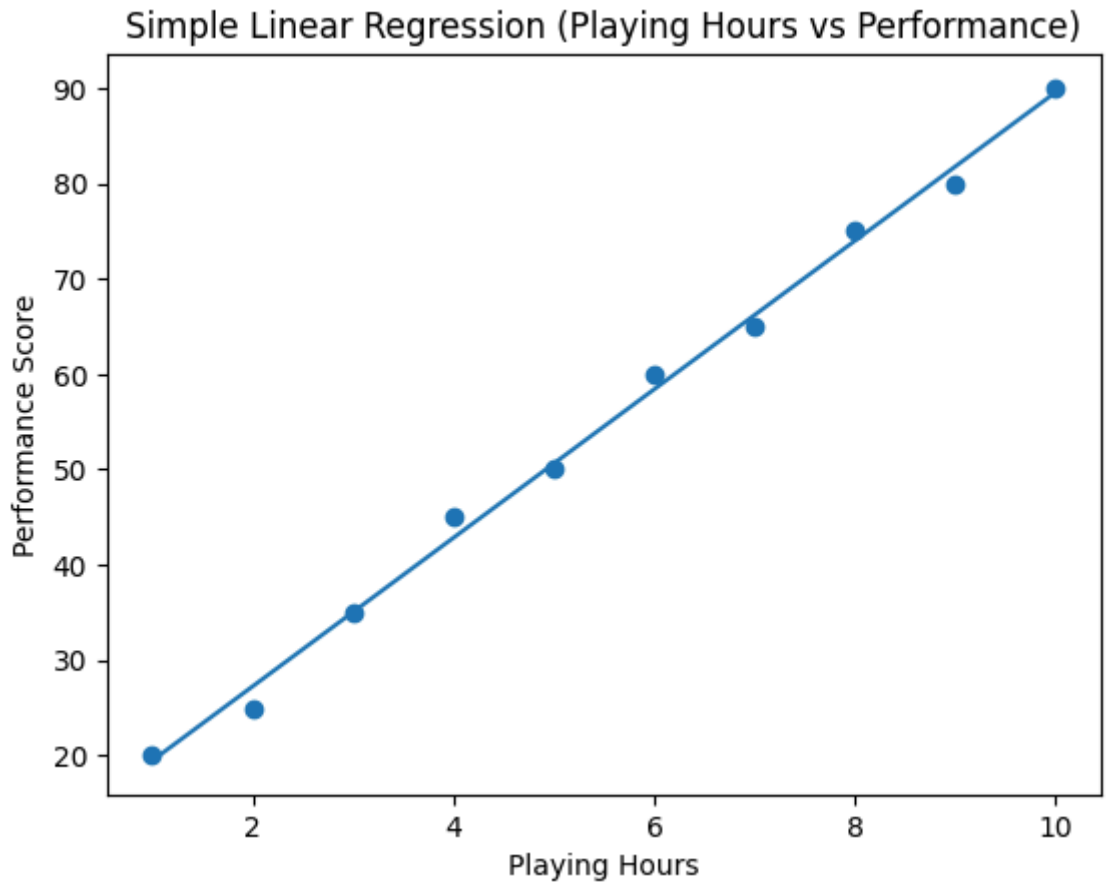
CODE:

Python

```python
# Simple Linear Regression using Least Squares Method

# Dataset: Playing Hours vs Performance

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

# Step 1: Load dataset

data = pd.read_csv("playing_data.csv")


# Step 2: Extract variables

x = data["PlayingHours"].values

y = data["Performance"].values


# Step 3: Calculate means

x_mean = np.mean(x)

y_mean = np.mean(y)


# Step 4: Calculate slope (b1)

numerator = np.sum((x - x_mean) * (y - y_mean))

denominator = np.sum((x - x_mean) ** 2)

b1 = numerator / denominator
```

```
# Step 5: Calculate intercept (b0)

b0 = y_mean - b1 * x_mean


print("Slope (b1):", b1)

print("Intercept (b0):", b0)

print("Regression Equation: y =", round(b0,2), "+", round(b1,2), "x")

# Step 6: Predicted values

y_pred = b0 + b1 * x

# Step 7: Plot graph

plt.figure()

plt.scatter(x, y)

plt.plot(x, y_pred)

plt.xlabel("Playing Hours")

plt.ylabel("Performance Score")

plt.title("Simple Linear Regression (Playing Hours vs Performance)")


plt.show()
```

OUTPUT:



Slope (b1): 7.787878787878788

Intercept (b0): 11.666666666666664

Regression Equation: y = 11.67 + 7.79 x

RESULT:

The Simple Linear Regression model was implemented successfully. The regression equation was obtained, and the graph shows a linear relationship between Playing Hours and Performance.

EXPERIMENT: 3

A PYTHON PROGRAM TO IMPLEMENT LOGISTIC MODEL

Aim:

To implement a Logistic Regression model using Python to predict the probability of passing an exam based on study hours.

Procedure:

1. Create a dataset `exam_logistic_data.csv` containing StudyHours and ExamResult.
2. Import required Python libraries such as pandas, numpy, and matplotlib.
3. Load the dataset using pandas.
4. Extract StudyHours as input variable (x) and ExamResult as output variable (y).
5. Initialize parameters b_0 (intercept), b_1 (slope), learning rate, and number of epochs.
6. Define the sigmoid function to convert linear output into probability.
7. Apply Gradient Descent to update model parameters.
8. Compute predicted probabilities using the sigmoid function.
9. Print the final values of b_0 and b_1.
10. Plot the sigmoid curve along with dataset points.

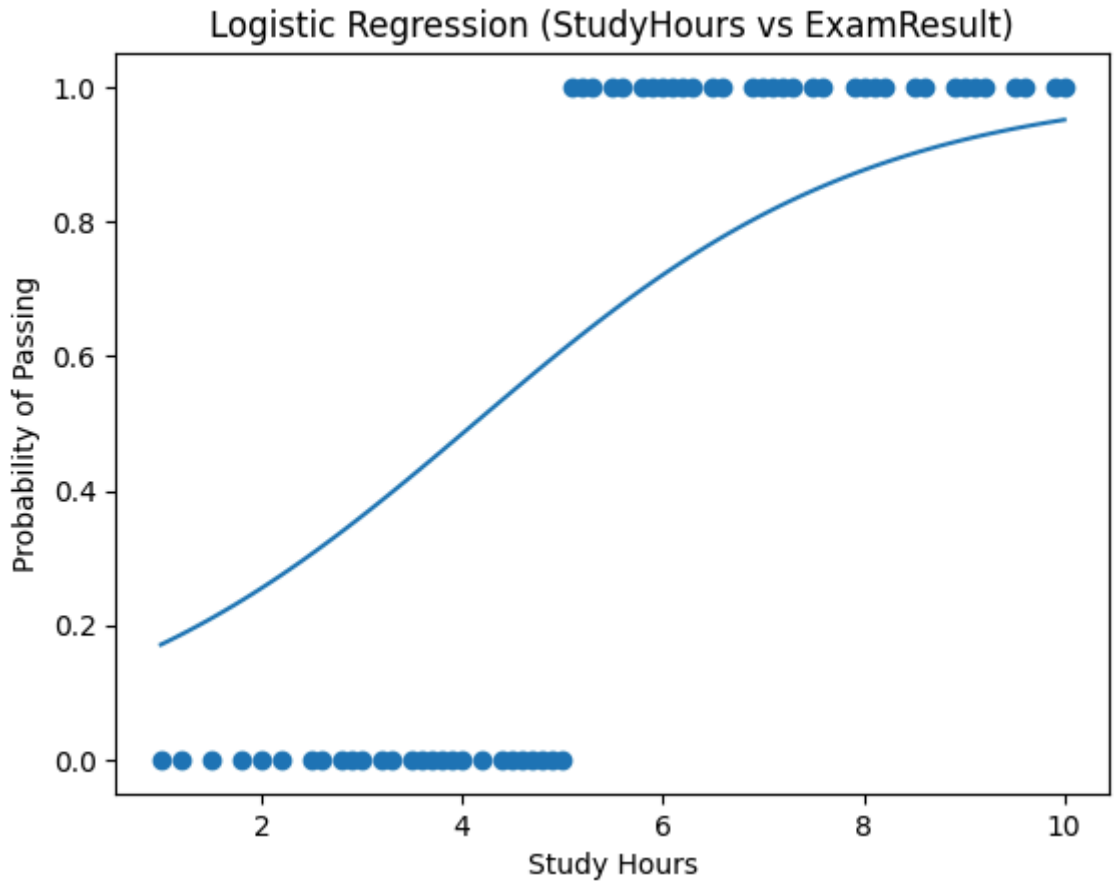
CODE:

```python
# Logistic Regression using exam_logistic_data.csv

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt
```

```
# Step 1: Load dataset

data = pd.read_csv("exam_logistic_data.csv")

# Step 2: Extract variables

x = data["StudyHours"].values

y = data["ExamResult"].values

# Step 3: Initialize parameters

b0 = 0

b1 = 0

learning_rate = 0.01

epochs = 2000


# Step 4: Sigmoid function

def sigmoid(z):

    return 1 / (1 + np.exp(-z))

# Step 5: Gradient Descent

for i in range(epochs):

    z = b0 + b1 * x

    y_pred = sigmoid(z)

    db0 = np.mean(y_pred - y)

    db1 = np.mean((y_pred - y) * x)


    b0 -= learning_rate * db0

    b1 -= learning_rate * db1
```

```
print("Intercept (b0):", round(b0,4))

print("Slope (b1):", round(b1,4))

# Step 6: Plot Sigmoid Curve

x_line = np.linspace(min(x), max(x), 100)

y_line = sigmoid(b0 + b1 * x_line)

plt.figure()

plt.scatter(x, y)

plt.plot(x_line, y_line)


plt.xlabel("Study Hours")

plt.ylabel("Probability of Passing")

plt.title("Logistic Regression (StudyHours vs ExamResult)")

plt.show()

# Step 6: Predicted values

y_pred = b0 + b1 * x

# Step 7: Plot graph

plt.figure()

plt.scatter(x, y)

plt.plot(x, y_pred)

plt.xlabel("Playing Hours")

plt.ylabel("Performance Score")

plt.title("Simple Linear Regression (Playing Hours vs Performance)")

plt.show()
```

OUTPUT:




```
Intercept (b0): -2.0836

Slope (b1): 0.5053
```

RESULT:

The Simple Linear Regression model was implemented successfully. The regression equation was obtained, and the graph shows a linear relationship between Playing Hours and Performance.

EXPERIMENT: 4

A PYTHON PROGRAM TO IMPLEMENT SINGLE LAYER PERCEPTRON.

Aim:

To implement a Single Layer Perceptron using Python to perform binary classification using the AND gate dataset.

Procedure:

1. Import the NumPy library for numerical operations.

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

Where:

- $x_1, x_2, ...x_n$ = input values
- $w_1, w_2, ...w_n$ = weights
- b = bias
- z = weighted sum

2. Define the input dataset representing the AND gate combinations.

3. Define the corresponding target output values.

Activation Function (Step Function)

$$y = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

4. Initialize the weights and bias to zero.

5. Set the learning rate and number of epochs for training.

6. Calculate the linear output using the dot product of inputs and weights.

7. Apply the step activation function to obtain predicted output.

8. Calculate the error between actual output and predicted output.
9. Update weights and bias using the perceptron learning rule.
10. Repeat the process for the specified number of epochs.
11. Print the final weights, bias, and predicted outputs.

CODE:

```python
import numpy as np


# Input dataset (AND gate)
X = np.array([[0,0],
              [0,1],
              [1,0],
              [1,1]])


# Output labels
y = np.array([0,0,0,1])


# Initialize weights and bias
weights = np.zeros(2)
bias = 0
```

```
learning_rate = 0.1
epochs = 10

# Training
for epoch in range(epochs):
    for i in range(len(X)):
        linear_output = np.dot(X[i], weights) + bias

        # Step Activation Function
        if linear_output >= 0:
            y_pred = 1
        else:
            y_pred = 0

        # Update rule
        error = y[i] - y_pred
        weights += learning_rate * error * X[i]
        bias += learning_rate * error

print("Final Weights:", weights)
print("Final Bias:", bias)

# Testing
```

```
for i in range(len(X)):
    linear_output = np.dot(X[i], weights) + bias
    if linear_output >= 0:
        prediction = 1
    else:
        prediction = 0
    print("Input:", X[i], "Prediction:", prediction)
```

OUTPUT:

```
Final Weights: [0.2 0.1]
Final Bias: -0.20000000000000004
Input: [0 0] Prediction: 0
Input: [0 1] Prediction: 0
Input: [1 0] Prediction: 0
Input: [1 1] Prediction: 1
```

RESULT:

The Single Layer Perceptron model was successfully implemented using Python.

EXPERIMENT: 5

A python program to implement multi layer perceptron with back propagation

Aim:

To implement a Multilayer Perceptron (MLP) model using a given dataset and to classify the output based on input features using a neural network.

Procedure:

1. Import required libraries and load the dataset.
2. Split data into features (X) and target (y), then apply train-test split.
3. Normalize data using feature scaling.
4. **Forward Propagation:**
   Compute outputs using:
   - $Z = WX + b$
   - $A = f(Z)$ (activation function like ReLU/Sigmoid)
5. **Backpropagation:**
   - Error = $y - \hat{y}$
   - Update weights:
     $W = W + \eta \cdot \nabla W$
6. Train the MLP model using training data.
7. Predict output for test data.
8. Evaluate using accuracy and confusion matrix.

CODE:

Python

```python
# ==============================

# STEP 1: Import Libraries

# ==============================

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt


from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.neural_network import MLPClassifier

from sklearn.metrics import accuracy_score, confusion_matrix


# ==============================

# STEP 2: Load Dataset

# ==============================

data = pd.read_csv("MLP.csv")


print("Dataset Preview:")

print(data.head())
```

```
# ==============================
# STEP 3: Split Features & Target
# (Last column = output)
# ==============================
X = data.iloc[:, :-1]
y = data.iloc[:, -1]

# ==============================
# STEP 4: Train-Test Split
# ==============================
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# ==============================
# STEP 5: Feature Scaling
# ==============================
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ==============================
# STEP 6: MLP Model
# ==============================
mlp = MLPClassifier(hidden_layer_sizes=(6,6),
                    activation='relu',
```

```
                    learning_rate_init=0.01,
                    max_iter=500)

# ==============================
# STEP 7: Training
# ==============================
mlp.fit(X_train, y_train)

# ==============================
# STEP 8: Prediction
# ==============================
y_pred = mlp.predict(X_test)

# ==============================
# STEP 9: Results
# ==============================
print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

# ==============================
# STEP 10: Visualization - Predictions
# ==============================
plt.figure()
plt.scatter(X_test[:,0], X_test[:,1], c=y_pred)
```

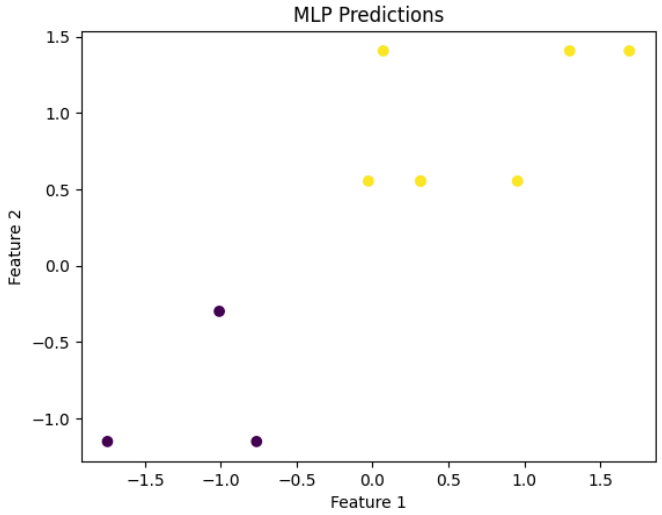
```
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("MLP Predictions")
plt.show()


# ==============================
# STEP 11: Loss Curve
# ==============================
plt.figure()
plt.plot(mlp.loss_curve_)
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.title("Loss Curve")
plt.show()
```

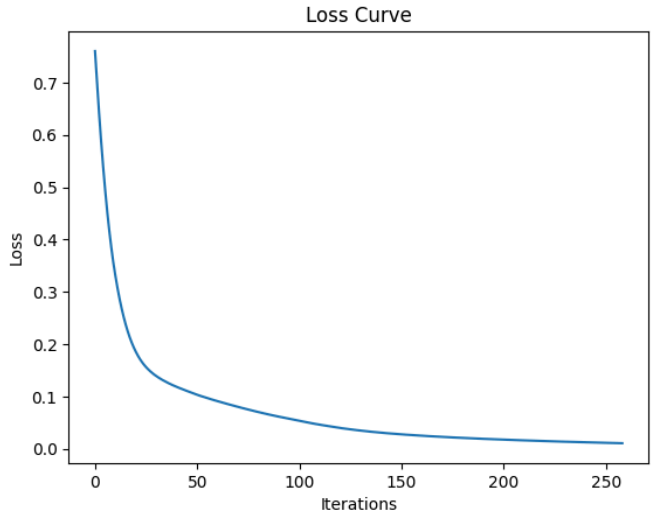
OUTPUT:

Dataset Preview:

	StudyHours	SleepHours	PracticeTests	ExamResult
0	1.0	5	0	0
1	2.0	6	1	0
2	1.5	5	0	0
3	3.0	6	1	0
4	2.5	6	1	0

```
Accuracy: 1.0

Confusion Matrix:

 [[3 0]

 [0 7]]
```





The decreasing loss curve indicates that the model learned effectively from the dataset

RESULT:

The Multilayer Perceptron (MLP) model was successfully trained and achieved good accuracy in classifying the data.

EXPERIMENT: 6

A PYTHON PROGRAM TO DO FACE RECOGNITION USING SVM CLASSIFIER.

Aim:

To implement face recognition using Support Vector Machine (SVM) classifier.

Procedure:

1. Import required libraries.
2. Load face dataset (Olivetti dataset).
3. Split dataset into training and testing sets.
4. Train the SVM classifier.
5. Predict test data using the trained model.
6. Evaluate accuracy.

CODE:

Rust

```
# Install (already available in Colab usually)

!pip install scikit-learn



# ==============================

# STEP 1: Import Libraries

# ==============================

import numpy as np

import matplotlib.pyplot as plt
```

```
from sklearn.datasets import fetch_olivetti_faces

from sklearn.svm import SVC

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score


# ==============================

# STEP 2: Load Online Dataset

# ==============================

faces = fetch_olivetti_faces()


X = faces.data       # flattened images

y = faces.target     # labels (0–39)


# ==============================

# STEP 3: Train-Test Split

# ==============================

X_train, X_test, y_train, y_test = train_test_split(

   X, y, test_size=0.2, random_state=42

)
```

```
# ==============================

# STEP 4: Train SVM

# ==============================

model = SVC(kernel='linear')

model.fit(X_train, y_train)



# ==============================

# STEP 5: Predict & Accuracy

# ==============================

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))



# ==============================

# STEP 6: Show Predictions

# ==============================

for i in range(5):

   plt.imshow(X_test[i].reshape(64,64), cmap='gray')

   plt.title(f"Predicted: {y_pred[i]}")

   plt.axis('off')

   plt.show()
```

OUTPUT:

```
Accuracy: 0.975
...
```

RESULT: The SVM classifier successfully recognized faces with good accuracy.

EXPERIMENT: 7

A PYTHON PROGRAM TO IMPLEMENT DECISION TREE

Aim:

To implement a Decision Tree classifier for classification.

Procedure:

1. Import necessary libraries.
2. Load dataset (Iris dataset).
3. Split data into training and testing sets.
4. Train Decision Tree model.
5. Perform prediction on test data.
6. Evaluate accuracy.

CODE:

Python

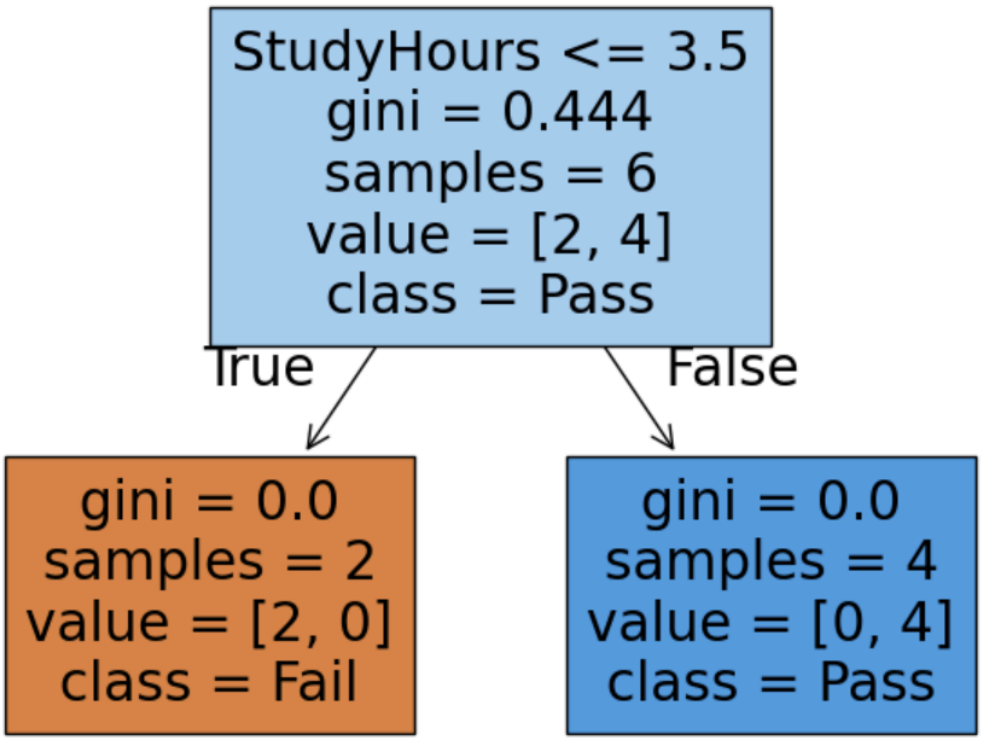
```
# ==============================

# STEP 1: Import Libraries

# ==============================

from sklearn.tree import plot_tree

import matplotlib.pyplot as plt



plt.figure(figsize=(8,6))

plot_tree(model,     feature_names=X.columns,     class_names=['Fail','Pass'],
filled=True)

plt.show()
```

```
# ==============================
# STEP 2: Create Simple Dataset
# ==============================
data = {
   'StudyHours': [1,2,3,4,5,6,7,8],
   'SleepHours': [5,6,6,7,7,8,8,9],
   'Pass':       [0,0,0,1,1,1,1,1]
}

df = pd.DataFrame(data)

# Features & Target
X = df[['StudyHours', 'SleepHours']]
y = df['Pass']

# ==============================
# STEP 3: Train-Test Split
# ==============================
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
```

```
)


# ==============================
# STEP 4: Train Decision Tree
# ==============================
model = DecisionTreeClassifier()
model.fit(X_train, y_train)


# ==============================
# STEP 5: Prediction
# ==============================
y_pred = model.predict(X_test)


# ==============================
# STEP 6: Accuracy
# ==============================
print("Accuracy:", accuracy_score(y_test, y_pred))
```

OUTPUT:

‘‘‘




```
Accuracy: 1.0
```

RESULT:

The Decision Tree model classified the data correctly with satisfactory accuracy.

EXPERIMENT: 8

A PYTHON PROGRAM TO IMPLEMENT BOOSTING

Aim:

To implement Boosting using AdaBoost algorithm.

Procedure:

1. Import required libraries.
2. Load dataset (Breast Cancer dataset).
3. Split dataset into training and testing sets.
4. Train AdaBoost classifier.
5. Predict output for test data.
6. Calculate accuracy.

CODE:

Python

```
# ==============================

# STEP 1: Import Libraries

# ==============================

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.ensemble import AdaBoostClassifier

from sklearn.metrics import accuracy_score
```

```
# ==============================
# STEP 2: Load Dataset
# ==============================
data = load_breast_cancer()

X = data.data
y = data.target

# ==============================
# STEP 3: Train-Test Split
# ==============================
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)

# ==============================
# STEP 4: Apply Boosting (AdaBoost)
# ==============================
model = AdaBoostClassifier(n_estimators=50)
```

```
model.fit(X_train, y_train)



# ==============================

# STEP 5: Prediction

# ==============================

y_pred = model.predict(X_test)



# ==============================

# STEP 6: Accuracy

# ==============================

print("Accuracy:", accuracy_score(y_test, y_pred))
```

OUTPUT:

```
Accuracy: 0.9649122807017544
```

RESULT:

The AdaBoost model improved classification accuracy by combining multiple weak learners.

EXPERIMENT: 9

A PYTHON PROGRAM TO IMPLEMENT KNN AND K-MEANS.

Aim:

To implement K-Nearest Neighbors (KNN) and K-Means clustering.

Procedure:

- **KNN:**

1. Import libraries.
2. Load dataset (Iris dataset).
3. Split into training and testing data.
4. Train KNN classifier.
5. Predict and evaluate accuracy.

- **K-Means:**

1. Import libraries.
2. Load dataset.
3. Apply K-Means clustering.
4. Assign clusters to data points.
5. Visualize clusters.

CODE:

Python

```
KNN

# ==============================

# STEP 1: Import Libraries

# ==============================
```

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score


# ==============================
# STEP 2: Load Dataset
# ==============================
data = load_iris()
X = data.data
y = data.target


# ==============================
# STEP 3: Train-Test Split
# ==============================
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)


# ==============================
```

```
# STEP 4: Train KNN
# ==============================
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)


# ==============================
# STEP 5: Prediction
# ==============================
y_pred = model.predict(X_test)


# ==============================
# STEP 6: Accuracy
# ==============================
print("KNN Accuracy:", accuracy_score(y_test, y_pred))
```

K-Means

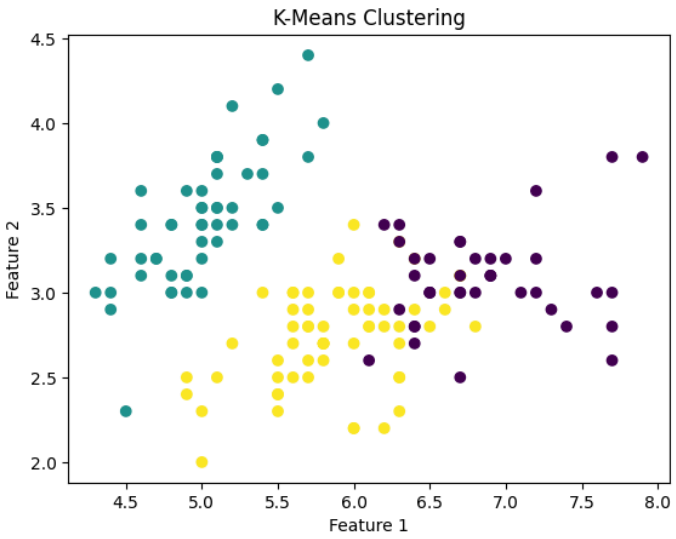
```
# ==============================
# STEP 1: Import Libraries
# ==============================
from sklearn.datasets import load_iris
```

```python
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt


# ==============================
# STEP 2: Load Dataset
# ==============================
data = load_iris()
X = data.data


# ==============================
# STEP 3: Apply K-Means
# ==============================
model = KMeans(n_clusters=3, random_state=42)
model.fit(X)


labels = model.labels_


# ==============================
# STEP 4: Visualize (2 features)
# ==============================
```

```
plt.scatter(X[:, 0], X[:, 1], c=labels)

plt.xlabel("Feature 1")

plt.ylabel("Feature 2")

plt.title("K-Means Clustering")

plt.show()
```

OUTPUT:

```
KNN Accuracy: 1.0
```



RESULT:

KNN successfully classified the data, and K-Means grouped the data into clusters effectively.

EXPERIMENT: 10

A PYTHON PROGRAM TO IMPLEMENT DIMENSIONALITY REDUCTION – PCA

Aim:

To implement dimensionality reduction using Principal Component Analysis (PCA).

Procedure:

1. Import required libraries.
2. Load dataset (Iris dataset).
3. Apply PCA to reduce dimensions.
4. Transform data into principal components.
5. Visualize reduced data.

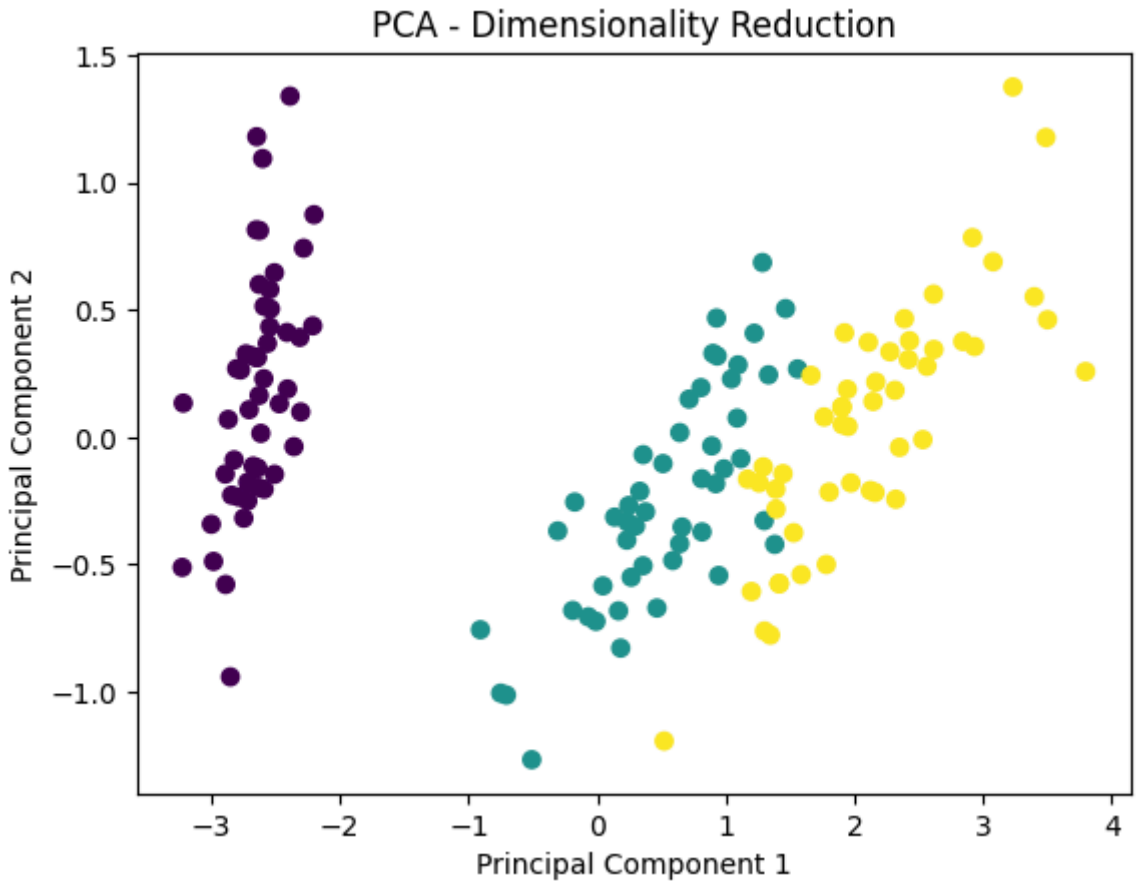
CODE:

Python

```python
# ==============================

# STEP 1: Import Libraries

# ==============================

from sklearn.datasets import load_iris

from sklearn.decomposition import PCA

import matplotlib.pyplot as plt



# ==============================

# STEP 2: Load Dataset
```

```
# ==============================

data = load_iris()

X = data.data

y = data.target



# ==============================

# STEP 3: Apply PCA (reduce to 2D)

# ==============================

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X)



# ==============================

# STEP 4: Visualize Result

# ==============================

plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y)

plt.xlabel("Principal Component 1")

plt.ylabel("Principal Component 2")

plt.title("PCA - Dimensionality Reduction")

plt.show()
```

OUTPUT:



RESULT:

PCA successfully reduced the dataset dimensions while preserving important information.